

Async Research Assistant

Topic 4 — AI Engineering Final Project

Team ExperimentMice · v1.0-final

Wikipedia

arXiv

Web Search



Concurrent fetch — Wikipedia · arXiv · Web



Exponential backoff retries + rate limiting



Graceful degradation — partial results still synthesized



TTL cache + optional PostgreSQL history

github.com/abdurahmanligulnisa/research-assistant · Due May 23, 2026



87%

Test Coverage

127

Tests Passing

2.7×

Concurrent Speedup

14.9s

Live-Net Concurrent

Project Overview

A software-engineering wrapper around an async AI research pipeline

What the system does

01

User submits a research question via CLI or Streamlit Web UI

02

System concurrently fetches Wikipedia, arXiv & web sources

03

Results cached by (source, canonicalized_query) key with TTL

04

Duplicate URLs deduped — first-seen order preserved

05

LLM synthesizes a cited Markdown answer (thread-pool offload)

06

Session persisted to PostgreSQL; graceful fallback if absent

Key Design Decisions



ai.* always via AIService

No direct imports in business logic — enables mocking, retry, and logging



asyncio.to_thread for synthesis

LLM call is synchronous; offloaded so the event loop is never blocked



Graceful degradation

One failing source ≠ total failure; answer produced from remaining sources

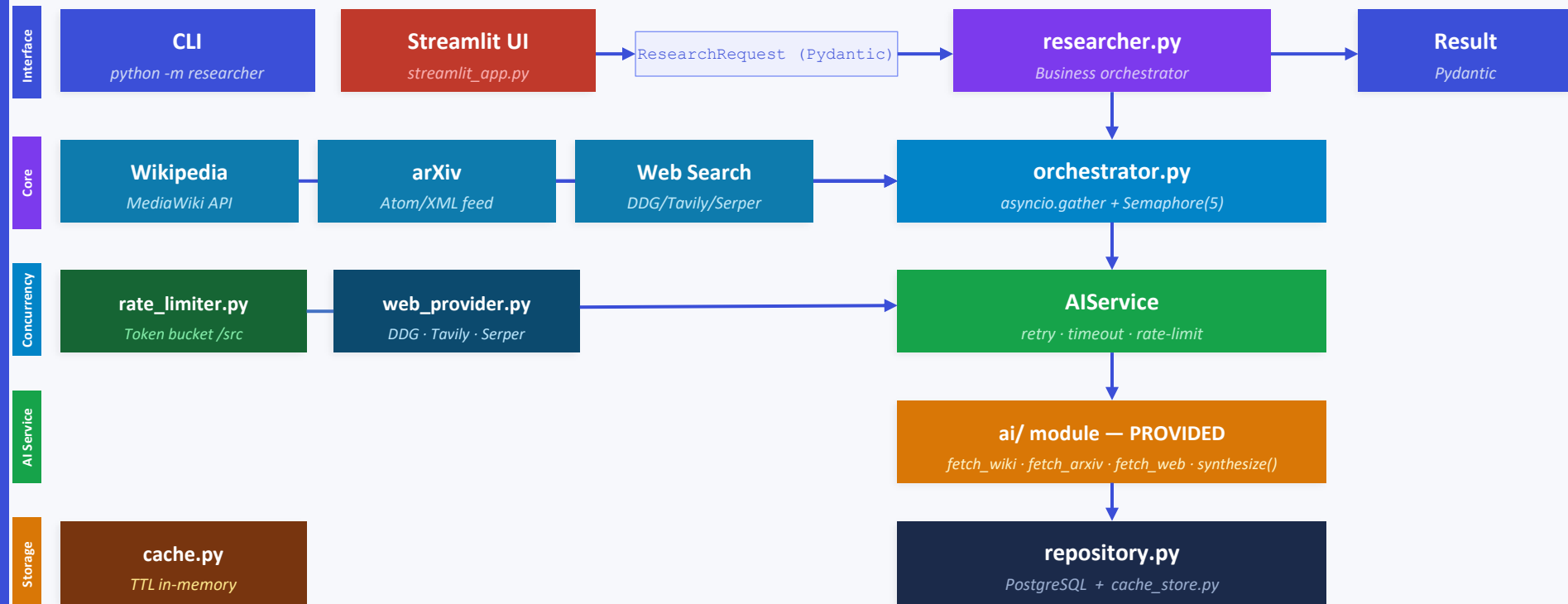


Cache keyed by canonical query

Lowercased + whitespace-normalized key prevents redundant fetches

System Architecture

End-to-end pipeline — bounded concurrency, retry, graceful degradation



Technology Stack

Python 3.12 (Docker) / 3.13 (local) · fully pinned requirements.txt



Python 3.12 / 3.13

Language



Claude / GPT-4o / Gemini

LLM (pluggable)



httpx + asyncio

Async HTTP



tenacity

Exp. Backoff



Token-bucket limiter

Rate Limiting



Pydantic v2 + settings

Validation



Click

CLI



Streamlit

Web UI (bonus)



psycopg3 + PostgreSQL

Database



pytest + respx + httpx

Testing



ruff + mypy

Lint / Types



Docker + Compose

Container

Concurrent Source Orchestration

src/concurrency/orchestrator.py — asyncio.gather + Semaphore + per-source timeout

orchestrator.py

```
async def fetch_sources_concurrent(
    question: str, sources: set[str] | None,
    max_concurrent: int | None = None,
) -> tuple[dict[str, list], list[str]]:

    limit = max_concurrent or settings.max_concurrent
    sem = asyncio.Semaphore(limit) # default=5

    async def _guarded(src_name: str):
        async with sem:
            return await _fetch_one(src_name, question,
                                    client=client)

    tasks = [_guarded(s) for s in sorted(active)]
    results = await asyncio.gather(
        *tasks, return_exceptions=True # partial ok
    )
```



Bounded Concurrency

`Semaphore(MAX_CONCURRENT=5)` prevents API flooding and rate-limit hits from parallel callers.



Graceful Degradation

`return_exceptions=True` — one failed source never aborts the run; partial results are synthesized.



Per-Source Timeout

`asyncio.wait_for()` enforces a hard wall-clock budget per source (wiki 4x, web 6x per-attempt).



Source Filtering

`--sources wiki,arxiv` limits `active_set`; missing sources skip gracefully with no error raised.

Retry & Rate-Limit Strategy

src/services/ai_service.py — tenacity decorator · RateLimitError · token-bucket per source

ai_service.py

```
class RateLimitError(ProviderError):
    # Carries Retry-After hint parsed from header
    def __init__(self, msg, retry_after=None):
        super().__init__(msg)
        self.retry_after = retry_after

def _make_retry(max_attempts=4):
    def _before_sleep(retry_state):
        exc = retry_state.outcome.exception()
        if isinstance(exc, RateLimitError) and \
            exc.retry_after is not None:
            # honour server Retry-After header exactly
            retry_state.next_action.sleep = exc.retry_after
    return retry(
        wait=wait_exponential(min=1, max=30),
        stop=stop_after_attempt(max_attempts),
        before_sleep=_before_sleep, reraise=True)
```

Retried (**_TRANSIENT**)

- ConnectionError
- TimeoutError
- OSError
- ProviderError (+ RateLimitError)
- RateLimitExceeded (token-bucket)

Token-Bucket Rate Limiters

Wikipedia	10 req/s
arXiv	2 req/s
Web search	1 req/s

Opt-in — callers wrap fetches with wikipedia_limiter, arxiv_limiter, web_limiter

Cache Design & Pydantic Models

src/services/cache.py · src/storage/cache_store.py · src/models.py

TTL In-Memory Cache

Key: (source_name, canonical_query)

Value: (timestamp: float, list[Source])



Lazy TTL eviction on get (default 3600s)



--no-cache bypasses both read AND write



CacheBackend ABC → swap Redis/JSON easily



Canonical: lower + strip + whitespace-norm



Ephemeral: wiped on restart, per-process

Pydantic v2 Models (models.py)

ResearchRequest

```
question: str (min=1, max=500)
sources: list[SourceFilter] | None
no_cache: bool = False
```

ResearchResult

```
question / answer: str
citations: list[CitationRecord]
sources_used / sources_failed
cached: bool · elapsed_seconds: float
```

CitationRecord

```
index: int · title · url · origin
```

Benchmark — Sequential vs Concurrent

scripts/bench.py · N=5 questions × 3 sources · Semaphore limit = 5

Offline Simulation

```
python scripts/bench.py --n 5 --offline
```

2.817s

Sequential · 1.0×

1.041s

Concurrent · 2.7×

Simulated provider latencies

Wikipedia 0.15 s

arXiv 0.20 s

Web search 0.18 s

Wall-clock $\approx \max(\text{wiki}, \text{arXiv}, \text{web})$ not sum

Live Network (Tavily)

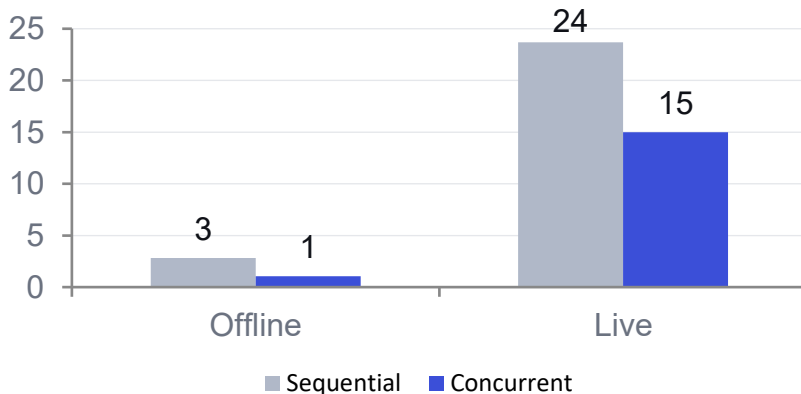
```
python scripts/bench.py --n 5
```

23.696s

Sequential · 1.0×

14.982s

Concurrent · 1.6×



Testing Strategy

127 tests · 87% coverage · fully offline · all HTTP mocked via respx / pytest-httpx

Coverage 87%



■ Covered (87%) ■ Uncovered (13%)

smoke	test_ai_smoke.py <i>PROVIDED contract — must always pass</i>	async	test_concurrency.py <i>gather, semaphore, graceful degradation</i>
cache	test_cache.py <i>TTL, canonical, expiry, clear</i>	cli	test_cli.py <i>Click CliRunner: ask, history, JSON</i>
svc	test_service.py <i>AIService retry, timeout, logging</i>	model	test_models.py <i>Pydantic validation edge cases</i>
db	test_storage.py <i>repository save / retrieve / error</i>	http	test_http_mocking.py <i>repx HTTP mocks, retry on conn-error</i>
db	test_repository.py <i>PostgreSQL repository (mocked)</i>	web	test_web_provider.py <i>DDG / Tavily / Serper adapters</i>
e2e	test_researcher.py <i>End-to-end pipeline (fully offline)</i>	log	test_logging_config.py <i>Logging setup, idempotent, level</i>

127

Tests

87%

Coverage

0

Net calls

✓

Smoke

conftest.py

FakeLLM · FakeWebSearch
no_real_ai · clear_cache (autouse)

CLI & Streamlit Web UI

src/cli.py (Click) · streamlit_app.py · researcher/ package

CLI · python -m researcher ask / history

```
# All three sources (default)
python -m researcher ask 'What is quantum entanglement?'
```

```
# Specific sources, bypass cache
python -m researcher ask 'CRISPR' \
    --sources wiki,arxiv --no-cache
```

```
# Machine-readable JSON for piping
python -m researcher ask 'AI safety' --json-output
```

```
# History (requires DATABASE_URL)
python -m researcher history --limit 10
```

```
# Verbose logging
python -m researcher --log-level DEBUG ask 'AI safety'
```



Streamlit Web UI (bonus)

```
streamlit run streamlit_app.py →
http://localhost:8501
```



Research question input with source selector



Cited Markdown answers with numbered refs



Session history — persisted via PostgreSQL



--no-cache toggle for always-fresh results



Docker single-container or full Compose stack

```
docker compose up --build → app + PostgreSQL
                           at :8501
```

Database & Storage Layer

src/storage/repository.py · psycopg3 async · PostgreSQL 14+

SQL schema · auto-created on init

```
CREATE TABLE IF NOT EXISTS research_sessions (  
    id SERIAL PRIMARY KEY,  
    question TEXT NOT NULL,  
    answer TEXT NOT NULL,  
    citations_json JSONB NOT NULL DEFAULT '[]',  
    sources_failed JSONB NOT NULL DEFAULT '[]',  
    cached BOOLEAN NOT NULL DEFAULT FALSE,  
    elapsed_seconds FLOAT NOT NULL DEFAULT 0.0,  
    created_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX IF NOT EXISTS idx_research_sessions_created_at  
ON research_sessions (created_at DESC);
```

`DATABASE_URL=postgresql://USER:PASS@HOST:5432/DBNAME` (omit → history
silently skipped)



repository.py features



AsyncConnectionPool — max_size=5 (env-cfg)



CREATE TABLE IF NOT EXISTS on first call



save_session() — full ResearchResult as JSONB



get_sessions(limit) → list[dict]



Graceful: warns + continues if DB absent



PostgreSQL-only: JSONB, SERIAL, TIMESTAMPTZ



SQLite not supported (psycopg3 %s placeholders)

Project Structure

github.com/abdurahmanligulnisa/research-assistant · tag v1.0-final

```
research-assistant/  
├── ai/                ← PROVIDED — DO NOT MODIFY  
│   ├── providers/    (Anthropic / OpenAI / Gemini)  
│   ├── schemas.py    Source, Citation, AnswerWithCitations  
│   ├── sources.py    fetch_wikipedia / fetch_arxiv / fetch_web  
│   └── synthesizer.py synthesize()  
├── src/              ← student SE layer  
│   ├── config.py     pydantic-settings  
│   ├── models.py     Pydantic models (Request / Result)  
│   ├── logging_config.py structured logging  
│   ├── services/     ai_service · cache · rate_limiter · web_provider  
│   ├── core/         researcher.py (full pipeline)  
│   ├── concurrency/  orchestrator.py  
│   ├── storage/      repository.py + cache_store.py  
│   └── cli.py        Click: ask / history  
├── researcher/       __main__.py (python -m researcher entry point)  
├── tests/            13 test files — 127 tests, 87% coverage  
├── scripts/          demo.py · bench.py · benchmark.py  
├── artefacts/        bench_results · coverage · pytest_output  
├── streamlit_app.py  ← Web UI (bonus +2)  
├── Dockerfile        ← multi-stage (bonus +1)  
├── docker-compose.yml ← app + PostgreSQL  
└── .github/workflows/ci.yml ← GitHub Actions (bonus +2)
```

Bonus Features — Deep Dive

Multi-stage Docker build · Custom token-bucket rate limiter · Non-root security



Multi-Stage Dockerfile (+1 bonus)

STAGE 1 — Builder

```
python:3.12-slim → pip install -r  
requirements.txt → /venv  
All build tools, compilers, cache — present here only
```



COPY /venv only

STAGE 2 — Runtime (final image)

```
python:3.12-slim → COPY --from=builder /venv  
/venv  
No build tools, no cache — significantly smaller image
```



Runs as non-root appuser — no root privileges in container

```
$ docker images researcher
```



Token-Bucket Rate Limiter

`src/services/rate_limiter.py` · custom implementation

Source	Default Rate
Wikipedia	10 req/s
arXiv	2 req/s
Web search	1 req/s

Integration with retry loop

`fetch_wikipedia(query)`

`RateLimitExceeded`
raised

`tenacity catches`
`_TRANSIENT`

`wait_exponential` →
retry

`RateLimitExceeded` is a subclass of the `_TRANSIENT` tuple — it is automatically caught and retried by the `tenacity` decorator in `AIService` with exponential backoff. No extra wiring needed once the limiter is enabled.

Known Limitations

Documented in README §14 and report/report.pdf §8



Ephemeral Cache

cache

In-process dict wiped on restart. Each worker has its own private cache. Fix: replace `_store` with Redis or filesystem-JSON via CacheBackend ABC.



PostgreSQL History Only

storage

History persisted only if `DATABASE_URL` is set. Absent or unreachable DB → silently skipped with a WARNING log. SQLite unsupported (psycopg3 syntax).



Bounded Connection Pool

db

`AsyncConnectionPool` caps at `max_size=5` (env-configurable). Heavy concurrent load queues inside the pool rather than opening new connections.



Rate Limiter Is Opt-In

api

`TokenBucketRateLimiter` (wiki 10/s, arXiv 2/s, web 1/s) exists in `rate_limiter.py` but is not auto-wired into `AIService`. Callers must opt in.



DuckDuckGo Rate-Limited

search

Burst traffic can return empty results or raise `RatelimitException`. Use `WEB_SEARCH_PROVIDER=tavily` (or `serper`) for production reliability.



No Cross-Process Sharing

scale

Multiple load-balanced workers each have an isolated cache island. Redis backend swap is straightforward via the CacheBackend ABC.

What We Built

Team ExperimentMice · AI-ENG-110 · May 2026



Full SE layer: config · cache · concurrency · retries · CLI · storage



127 tests, 87% coverage, zero network hits in CI — all mocked



Multi-stage Dockerfile — builder + runtime + non-root appuser



GitHub Actions CI workflow — tagged v1.0-final



Custom token-bucket rate limiter — wiki 10/s · arXiv 2/s · web 1/s



2.7× offline speedup · 1.6× live-network (Tavily, N=5)



Streamlit Web UI — question input, citations, session history

87%

Test Coverage

127

Tests Passing

2.7×

Offline Speedup

1.6×

Live-Network Speedup

+7 pts

Bonus
(Docker+CI+UI+Token-
bucket rate limiter)